



Logical & Special Operators

Mastering Advanced Operators in C Programming



Logical



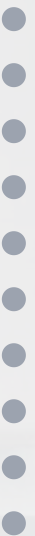
Increment



Conditional



Sizeof



Logical Operators



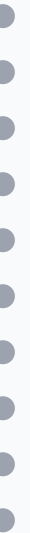
Used to evaluate expressions which may be **true or false**



Combine simple conditions to form **more complex conditions**



Instead of multiple IF statements, use logical operators to test multiple conditions at once!



Types of Logical Operators

&&

Logical AND

True if both operands are non-zero

`(A && B) is false`

||

Logical OR

True if any operand is non-zero

`(A || B) is true`

!

Logical NOT

Reverses the logical state

`!(A && B) is true`



Assumption: Variable A holds 1 (true) and variable B holds 0 (false)



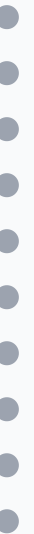
Truth Tables

Logical AND (&&) Truth Table

X	Y	X && Y
0	0	0
1	0	0
0	1	0
1	1	1

Logical OR (||) Truth Table

X	Y	X Y
0	0	0
1	0	1
0	1	1



X	Y	X Y
1	1	1

Logical NOT (!) Truth Table

X	!X
0	1
1	0



Logical Operator Examples

Example 1: Grade Range Check

Grade 'B' if marks are between 65 to 75:

```
if ((marks >= 65) && (marks <= 75))  
    printf("Grade is B\n");
```

Example 2: Admission Eligibility

Eligible if PCM > 85% OR aggregate > 90%:

```
if ((PCM >= 85) || (aggregate >= 90))  
    printf("Eligible for admission\n");
```

Example 3: Logical NOT

Check if grade is not 'A':

```
if (!(grade == 'A'))  
    printf("The next grade is %c\n", grade);
```



Note: Parentheses are needed because ! has higher precedence than ==



Increment and Decrement Operators



Two very useful operators **++** and **--** that add or subtract one from the operand

Prefix

Operator used before the operand:

`++A, --B`



Follows **F.C.T.U** (First Change Then Use)

Postfix

Operator used after the operand:

`A++, B--`



Follows **F.U.T.C** (First Use Then Change)



Important: These operators require only one operand (unary operators) and the operand must be a variable, not a constant!



Increment/Decrement Examples

Example 1: Post Increment

```
int N = 10, R;  
R = N++; // post increment
```

```
printf("R=%d \n N=%d", R, N);
```

Output: R=10, N=11

Value is assigned first, then incremented

Example 2: Complex Expression

```
int R, N = 10;  
R = ++N + --N + --N + N++ + --N;  
printf("R=%d \n N=%d", R, N);
```

Output: R=40, N=9

Execution order: postfix → operation → prefix



Execution Order: In complex expressions, all prefix operations execute first, then arithmetic operations, then postfix operations

? Conditional or Ternary Operator



C's only **ternary operator** - takes three operands and is closely related to the if/else structure

Syntax

```
(condition) ? (expression1) : (expression2);
```

If condition is true, expression1 is evaluated; else expression2 is evaluated

Example 1:

```
x = (y < 20) ? 9 : 10;
```

If $y < 20$, then $x = 9$ else $x = 10$

Example 2:

```
printf("%s\n", grade >= 50 ? "Passed" : "failed");
```

Prints "Passed" if $\text{grade} \geq 50$ else prints "failed"

Example 3:

```
(a > b) ? printf("a is greater than b\n") : printf("b is greater than a\n");
```

Executes first printf if a > b, else executes second printf

★ Special Operators

Comma Operator

Used to link related expressions together:

```
int val, x, y;  
value = (x = 10, y = 5, x + y);
```

 First assigns 10 to x, then 5 to y, finally sum x + y to value

Sizeof Operator

Returns the number of bytes an operand occupies:

```
int n;  
n = sizeof(int);  
printf("n=%d\n", n); // Output: n=2 or 4 (depends on system)  
  
n = sizeof(double);  
printf("n=%d", n); // Output: n=8
```

 A compile-time operator that works on variables, constants, or data types

 **Note:** Other special operators include pointer operators (& and *) and member selection operators (. and ->) which will be discussed later

Shorthand & Priority Operators

C Shorthand Operators

Simplifies coding of certain assignment statements:

Operator	Example	Meaning
+=	a += 2	$a = a + 2$
-=	a -= 2	$a = a - 2$
*=	a *= 2	$a = a * 2$
/=	a /= 2	$a = a / 2$
%=	a %= 2	$a = a \% 2$

Operator Precedence

Hierarchy for solving mixed expressions (highest to lowest):

() Parentheses	Left to right
! ++ -- (type) sizeof Unary	Right to left
* / % Multiplicative	Left to right
+ - Additive	Left to right
< <= > >= Relational	Left to right
== != Equality	Left to right
&& Logical AND	Left to right
 Logical OR	Left to right
?: Conditional	Right to left
= += -= *= /= %= Assignment	Right to left
, Comma	Left to right

